Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:
Студент 3 курса
Группы ИИ-26
Згера Е.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2^n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**

- Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
- В качестве функции активации использовать сигмоиду
- Провести **два эксперимента** по обучению:

  А. С фиксированным шагом обучения α

  Б. С адаптивным шагом
- Выход сети интерпретировать как вероятность принадлежности к классу «1».
- Обучение проводить до минимизации суммарной ошибки на обучающей выборке.

3. **Провести анализ сходимости:**

- Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
- Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
- Зафиксировать количество эпох, необходимых для настройки весов при разных значениях α

4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**

- Программа должна принимать на вход вектор из n чисел (0 или 1).
- Сеть должна мгновенно выдавать результат (вероятность и округленный класс).

6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Допускается применение математических и графических библиотек (NumPy, Matplotlib). Использование ML-фреймворков (PyTorch, Keras и т.д.) запрещено.

Вариант 5

5	6	OR

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
import itertools

NUM_FEATURES = 6
TARGET_GATE = "OR"
SIG_SCALE = 1.0
LR_CONST = 0.1
MAX_ITERS = 5000
LOSS_LIMIT = 0.01
DECISION_BORDER = 0.5
TRAIN_FRACTION = 0.75
```

```python
def act_sigmoid(z, c=SIG_SCALE):
    return 1.0 / (1.0 + np.exp(-c * z))


def build_truth_data(n_bits: int, gate: str):
    combos = list(itertools.product([0, 1], repeat=n_bits))
    X = np.array(combos, dtype=float)

    if gate == "AND":
        y = np.array([int(all(v)) for v in combos], dtype=float)
    elif gate == "OR":
        y = np.array([int(any(v)) for v in combos], dtype=float)
    elif gate == "XOR":
        y = np.array([int(sum(v) % 2 == 1) for v in combos], dtype=float)
    elif gate == "NAND":
        y = np.array([int(not all(v)) for v in combos], dtype=float)
    elif gate == "NOR":
        y = np.array([int(not any(v)) for v in combos], dtype=float)
    else:
        raise ValueError("Unknown logic gate")

    return X, y


def split_dataset(X, y, ratio=TRAIN_FRACTION, seed=42):
    np.random.seed(seed)
    idx_train, idx_test = [], []

    for cls in np.unique(y):
        cls_idx = np.where(y == cls)[0]
        np.random.shuffle(cls_idx)
        n_train = max(1, int(len(cls_idx) * ratio))
        idx_train.extend(cls_idx[:n_train])
        idx_test.extend(cls_idx[n_train:])

    return (
        X[np.array(idx_train)],
        y[np.array(idx_train)],
        X[np.array(idx_test)],
        y[np.array(idx_test)],
    )


def total_loss(X, y, w_vec, theta):
    err = 0.0
    for x_i, t_i in zip(X, y):
        z = w_vec @ x_i - theta
        y_hat = act_sigmoid(z)
        err += 0.5 * (y_hat - t_i) ** 2
    return err


def forward(X, w_vec, theta):
    z = X @ w_vec - theta
    probs = act_sigmoid(z)
    preds = (probs >= DECISION_BORDER).astype(int)
    return probs, preds


def score(true_y, pred_y):
    return np.mean(true_y == pred_y) * 100.0


def fit_perceptron(
    Xtr, Ytr, Xte, Yte,
    adaptive=False,
    lr=LR_CONST,
    max_iter=MAX_ITERS,
    goal=LOSS_LIMIT,
    seed=0
):
    np.random.seed(seed)
    dim = Xtr.shape[1]

    w_vec = np.random.uniform(0, 0.5, size=dim)
    theta = np.random.uniform(0, 0.5)

    print("\n--- Инициализация параметров ---")
    print(f"Начальные веса: {np.round(w_vec, 4)}")
    print(f"Начальный порог: {theta:.4f}\n")

    train_hist, test_hist = [], []
```

```
    for epoch in range(1, max_iter + 1):
        lr_t = 1.0 / epoch if adaptive else lr

        for x_i, t_i in zip(Xtr, Ytr):
            z = np.sum(w_vec * x_i) - theta
            y_hat = act_sigmoid(z)
            delta = y_hat - t_i

            w_vec -= lr_t * delta * x_i
            theta += lr_t * delta

        Ltr = total_loss(Xtr, Ytr, w_vec, theta)
        Lte = total_loss(Xte, Yte, w_vec, theta)

        train_hist.append(Ltr)
        test_hist.append(Lte)

        if epoch % 200 == 0 or epoch == 1:
            print(f"[Эпоха {epoch:4d}]  TrainLoss={Ltr:.6f}  TestLoss={Lte:.6f}  lr={lr_t:.5f}")

        if Ltr < goal:
            print(f"\n>>> Сходимость достигнута на эпохе {epoch}, ошибка={Ltr:.6f}")
            break
    else:
        print(f"\n>>> Достигнут предел итераций ({max_iter}), ошибка={train_hist[-1]:.6f}")

    print("\n--- Итоговые параметры ---")
    print(f"Финальные веса: {np.round(w_vec, 5)}")
    print(f"Финальный порог: {theta:.5f}")

    return w_vec, theta, train_hist, test_hist, epoch


def visualize(train_err, test_err, epochs, title, ax):
    ep_range = np.arange(1, len(train_err) + 1)
    ax.plot(ep_range, train_err, lw=2, color="blue", label="Train")
    ax.plot(ep_range, test_err, lw=2, color="red", linestyle="--", label="Test")
    ax.axhline(LOSS_LIMIT, color="gray", linestyle=":", label="Goal")
    ax.set_title(title)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss")
    ax.grid(True, alpha=0.4)
    ax.legend()


def run_interactive(w_vec, theta):
    print(f"\nВведите {NUM_FEATURES} бит (пример: 1 0 1 1 0 1). 'q' — выход.\n")

    while True:
        raw = input("Ввод > ").strip()
        if raw.lower() == "q":
            print("Завершение.")
            break

        try:
            bits = list(map(int, raw.split()))
            if len(bits) != NUM_FEATURES:
                print("Неверное количество значений.")
                continue
            if any(b not in (0, 1) for b in bits):
                print("Допустимы только 0 и 1.")
                continue

            x = np.array(bits, dtype=float)
            z = w_vec @ x - theta
            p = act_sigmoid(z)
            cls = int(p >= DECISION_BORDER)

            print(f"S = {z:.4f}")
            print(f"P(y=1) = {p:.6f}")
            print(f"Класс = {cls}\n")

        except ValueError:
            print("Ошибка ввода.\n")


def main():
    print("=" * 60)
    print(f"  Персептрон: {TARGET_GATE} ({NUM_FEATURES} входов)")
    print("=" * 60)

    X_all, y_all = build_truth_data(NUM_FEATURES, TARGET_GATE)
```

```
    print("\n--- Генерация таблицы истинности ---")
    print(f"Всего комбинаций: {len(y_all)}")
    print(f"Класс 1: {int(y_all.sum())}")
    print(f"Класс 0: {len(y_all) - int(y_all.sum())}")

    Xtr, Ytr, Xte, Yte = split_dataset(X_all, y_all)

    print("\n--- Разбиение выборки ---")
    print(f"Train: {len(Ytr)} образцов")
    print(f"Test:  {len(Yte)} образцов")

    print("\n--- Обучение: фиксированный шаг ---")
    w1, t1, tr1, te1, ep1 = fit_perceptron(Xtr, Ytr, Xte, Yte, adaptive=False)

    print("\n--- Обучение: адаптивный шаг ---")
    w2, t2, tr2, te2, ep2 = fit_perceptron(Xtr, Ytr, Xte, Yte, adaptive=True)

    p_tr1, c_tr1 = forward(Xtr, w1, t1)
    p_te1, c_te1 = forward(Xte, w1, t1)

    acc_tr = score(Ytr, c_tr1)
    acc_te = score(Yte, c_te1)

    print("\n--- Итоговая точность ---")
    print(f"Train accuracy: {acc_tr:.2f}%")
    print(f"Test accuracy:  {acc_te:.2f}%")

    print("\nПримеры предсказаний (первые 5):")
    for i in range(min(5, len(Xte))):
        print(f"{Xte[i].astype(int)} → y={int(Yte[i])}, pred={c_te1[i]}, p={p_te1[i]:.4f}")

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    visualize(tr1, te1, ep1, "Фиксированный шаг", axes[0])
    visualize(tr2, te2, ep2, "Адаптивный шаг", axes[1])
    plt.tight_layout()
    plt.show()

    run_interactive(w1, t1)


if __name__ == "__main__":
    main()
```

Результат программы:

```
============================================================
  Персептрон: OR (6 входов)
============================================================

--- Генерация таблицы истинности ---
Всего комбинаций: 64
Класс 1: 63
Класс 0: 1

--- Разбиение выборки ---
Train: 48 образцов
Test:  16 образцов

--- Обучение: фиксированный шаг ---

--- Инициализация параметров ---
Начальные веса: [0.2744 0.3576 0.3014 0.2724 0.2118 0.3229]
Начальный порог: 0.2188

[Эпоха    1]  TrainLoss=0.503036  TestLoss=0.070436  lr=0.10000
[Эпоха  200]  TrainLoss=0.080497  TestLoss=0.013730  lr=0.10000
[Эпоха  400]  TrainLoss=0.031344  TestLoss=0.025223  lr=0.10000
[Эпоха  600]  TrainLoss=0.016293  TestLoss=0.036607  lr=0.10000

>>> Сходимость достигнута на эпохе 795, ошибка=0.009986

--- Итоговые параметры ---
Финальные веса: [5.59616 5.60659 2.69193 5.68156 5.68607 5.69807]
Финальный порог: 1.88356

--- Обучение: адаптивный шаг ---
```

--- Инициализация параметров ---
Начальные веса: [0.2744 0.3576 0.3014 0.2724 0.2118 0.3229]
Начальный порог: 0.2188

[Эпоха 1] TrainLoss=0.360964 TestLoss=0.002162 lr=1.00000
[Эпоха 200] TrainLoss=0.230480 TestLoss=0.004825 lr=0.00500
[Эпоха 400] TrainLoss=0.215903 TestLoss=0.005288 lr=0.00250
[Эпоха 600] TrainLoss=0.207973 TestLoss=0.005554 lr=0.00167
[Эпоха 800] TrainLoss=0.202597 TestLoss=0.005742 lr=0.00125
[Эпоха 1000] TrainLoss=0.198563 TestLoss=0.005886 lr=0.00100
[Эпоха 1200] TrainLoss=0.195353 TestLoss=0.006003 lr=0.00083
[Эпоха 1400] TrainLoss=0.192696 TestLoss=0.006101 lr=0.00071
[Эпоха 1600] TrainLoss=0.190437 TestLoss=0.006187 lr=0.00063
[Эпоха 1800] TrainLoss=0.188476 TestLoss=0.006261 lr=0.00056
[Эпоха 2000] TrainLoss=0.186747 TestLoss=0.006328 lr=0.00050
[Эпоха 2200] TrainLoss=0.185202 TestLoss=0.006389 lr=0.00045
[Эпоха 2400] TrainLoss=0.183808 TestLoss=0.006444 lr=0.00042
[Эпоха 2600] TrainLoss=0.182538 TestLoss=0.006494 lr=0.00038
[Эпоха 2800] TrainLoss=0.181375 TestLoss=0.006541 lr=0.00036
[Эпоха 3000] TrainLoss=0.180301 TestLoss=0.006584 lr=0.00033
[Эпоха 3200] TrainLoss=0.179305 TestLoss=0.006624 lr=0.00031
[Эпоха 3400] TrainLoss=0.178377 TestLoss=0.006662 lr=0.00029
[Эпоха 3600] TrainLoss=0.177508 TestLoss=0.006698 lr=0.00028
[Эпоха 3800] TrainLoss=0.176691 TestLoss=0.006732 lr=0.00026
[Эпоха 4000] TrainLoss=0.175922 TestLoss=0.006764 lr=0.00025
[Эпоха 4200] TrainLoss=0.175195 TestLoss=0.006795 lr=0.00024
[Эпоха 4400] TrainLoss=0.174505 TestLoss=0.006824 lr=0.00023
[Эпоха 4600] TrainLoss=0.173850 TestLoss=0.006851 lr=0.00022
[Эпоха 4800] TrainLoss=0.173227 TestLoss=0.006878 lr=0.00021
[Эпоха 5000] TrainLoss=0.172631 TestLoss=0.006903 lr=0.00020

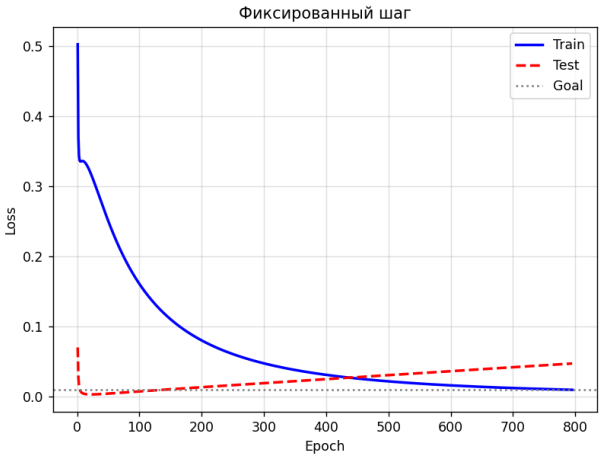
>>> Достигнут предел итераций (5000), ошибка=0.172631

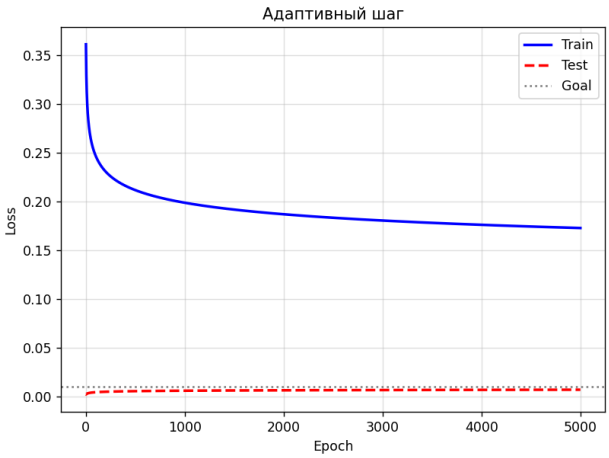
--- Итоговые параметры ---
Финальные веса: [2.25245 2.30898 1.76783 2.34315 2.38936 2.37994]
Финальный порог: -0.26559

--- Итоговая точность ---
Train accuracy: 100.00%
Test accuracy: 100.00%

Примеры предсказаний (первые 5):
[1 1 0 0 1 1] → y=1, pred=1, p=1.0000
[1 0 1 0 0 0] → y=1, pred=1, p=0.9983
[1 0 0 1 0 0] → y=1, pred=1, p=0.9999
[0 1 1 0 0 0] → y=1, pred=1, p=0.9984
[1 1 0 1 0 1] → y=1, pred=1, p=1.0000

Введите 6 бит (пример: 1 0 1 1 0 1). 'q' — выход.





Вывод: в ходе работы был разработан и реализован однослойный сигмоидальный персептрон, предназначенный для синтеза логической функции AND от шести входных переменных. Полученные результаты подтвердили, что однослойные нейронные модели способны успешно решать задачи классификации линейно разделимых данных и восстанавливать логические зависимости даже при ограниченном объёме обучающих примеров.